In [1]:
```
import json
import os
import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import Subset, TensorDataset, DataLoader
from sklearn.cluster import KMeans, MiniBatchKMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
```

In [2]:
```
cwd = os.getcwd()
project_root = os.path.dirname(cwd) if os.path.basename(cwd) == 'src' else c
config_path = os.path.join(project_root, 'config', 'config.json')

with open(config_path, 'r') as f:
    config = json.load(f)

config['dataset']['root'] = os.path.join(project_root, config['dataset']['rc
results_dir = os.path.join(project_root, 'results')
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(config['dataset']['mean'], config['dataset']['std']
])

cifar10_full = torchvision.datasets.CIFAR10(root=config['dataset']['root'],
pool_loader = DataLoader(cifar10_full, batch_size=config['feature_extractor'

testset = torchvision.datasets.CIFAR10(root=config['dataset']['root'], trair
test_loader = DataLoader(testset, batch_size=config['classifier']['test_batc
```

```
Using device: cuda
100%|██████████| 170M/170M [00:02<00:00, 68.7MB/s]
```

In [3]:
```
class SimCLR_ResNet18(nn.Module):
    def __init__(self):
        super(SimCLR_ResNet18, self).__init__()
        base_model = torchvision.models.resnet18(weights=None)
        base_model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding
        base_model.maxpool = nn.Identity()
        self.encoder = nn.Sequential(*list(base_model.children())[:-1])
        self.projector = nn.Sequential(
            nn.Linear(512, 512, bias=False),
            nn.Linear(512, 128, bias=False)
        )
```

```
    def forward(self, x):
        h = self.encoder(x).view(x.size(0), -1)
        z = self.projector(h)
        return h, z

model = SimCLR_ResNet18().to(device)
weights_path = os.path.join(project_root, 'models', 'simclr_resnet18_epoch_5
model.load_state_dict(torch.load(weights_path, map_location=device))
print("Loaded checkpoint.")
model.eval()

embeddings, true_labels = [], []
with torch.no_grad():
    for images, labels in pool_loader:
        features, _ = model(images.to(device))
        embeddings.append(F.normalize(features, p=2, dim=1).cpu().numpy())
        true_labels.append(labels.numpy())
embeddings = np.concatenate(embeddings, axis=0)
true_labels = np.concatenate(true_labels, axis=0)

test_embeddings, test_labels = [], []
with torch.no_grad():
    for images, labels in test_loader:
        features, _ = model(images.to(device))
        test_embeddings.append(F.normalize(features, p=2, dim=1).cpu().numpy
        test_labels.append(labels.numpy())
test_embeddings = np.concatenate(test_embeddings, axis=0)
test_labels = np.concatenate(test_labels, axis=0)

test_ds = TensorDataset(torch.tensor(test_embeddings, dtype=torch.float32),
test_dl = DataLoader(test_ds, batch_size=config['classifier']['test_batch_si
```

```
Loaded checkpoint.
```

In [4]:

```
def get_typiclust_queries(embeddings, B, max_clusters=500, noise_threshold=0
    N = embeddings.shape[0]

    nn_outlier = NearestNeighbors(n_neighbors=5).fit(embeddings)
    distances_outlier, _ = nn_outlier.kneighbors(embeddings)
    noise_scores = distances_outlier[:, -1]

    mask_absolute = noise_scores <= noise_threshold
    dynamic_threshold = np.percentile(noise_scores, 95)
    mask_percentile = noise_scores <= dynamic_threshold
    valid_mask = mask_absolute & mask_percentile

    clean_embeddings = embeddings[valid_mask]
    original_indices = np.where(valid_mask)[0]

    pca = PCA(n_components=min(pca_dims, clean_embeddings.shape[1]))
    reduced_embeddings = pca.fit_transform(clean_embeddings)

    if len(reduced_embeddings) < B:
        reduced_embeddings, original_indices = embeddings, np.arange(N)

    num_clusters = min(B, max_clusters)
```

```
    if num_clusters <= 50:
        kmeans = KMeans(n_clusters=num_clusters, random_state=config['typicl
    else:
        kmeans = MiniBatchKMeans(n_clusters=num_clusters, random_state=confi

    cluster_labels = kmeans.fit_predict(reduced_embeddings)
    clusters = {i: [] for i in range(num_clusters)}
    for idx, label in enumerate(cluster_labels): clusters[label].append(idx)

    valid_clusters = {k: v for k, v in clusters.items() if len(v) >= 5}
    top_b_clusters = sorted(valid_clusters.items(), key=lambda item: len(ite

    queries = []
    for cluster_id, indices in top_b_clusters:
        cluster_embeddings = reduced_embeddings[indices]
        nn_typ = NearestNeighbors(n_neighbors=min(20, len(indices))).fit(clu
        distances, _ = nn_typ.kneighbors(cluster_embeddings)
        typicality = -np.mean(distances, axis=1)

        best_clean_idx = indices[np.argmax(typicality)]
        queries.append(original_indices[best_clean_idx])
    return queries

def get_entropy_queries(embeddings, true_labels, B):
    seed_idx = np.random.choice(len(embeddings), min(10, B), replace=False)
    if B <= 10: return seed_idx.tolist()

    X_seed = torch.tensor(embeddings[seed_idx], dtype=torch.float32).to(devi
    y_seed = torch.tensor(true_labels[seed_idx], dtype=torch.long).to(device

    proxy_clf = nn.Linear(embeddings.shape[1], 10).to(device)
    opt = optim.SGD(proxy_clf.parameters(), lr=0.1)
    criterion = nn.CrossEntropyLoss()

    for _ in range(20):
        opt.zero_grad()
        loss = criterion(proxy_clf(X_seed), y_seed)
        loss.backward()
        opt.step()

    with torch.no_grad():
        all_X = torch.tensor(embeddings, dtype=torch.float32).to(device)
        probs = F.softmax(proxy_clf(all_X), dim=1)
        entropy = -torch.sum(probs * torch.log(probs + 1e-8), dim=1).cpu().r
    return np.argsort(entropy)[-B:].tolist()
```

In [5]:

```
import copy

class BasicBlock(nn.Module):
    def __init__(self, in_p, out_p, stride, drop=0.0, act_before=False):
        super().__init__()
        self.bn1, self.relu1 = nn.BatchNorm2d(in_p), nn.LeakyReLU(0.1, True)
        self.conv1 = nn.Conv2d(in_p, out_p, 3, stride, 1, bias=False)
        self.bn2, self.relu2 = nn.BatchNorm2d(out_p), nn.LeakyReLU(0.1, True
        self.conv2 = nn.Conv2d(out_p, out_p, 3, 1, 1, bias=False)
        self.drop, self.equal = drop, in_p == out_p
```

```
        self.shortcut = None if self.equal else nn.Conv2d(in_p, out_p, 1, st
        self.act_before = act_before
    def forward(self, x):
        if not self.equal and self.act_before:
            out = self.relu1(self.bn1(x))
        else:
            out = self.relu1(self.bn1(x))
        out = self.relu2(self.bn2(self.conv1(out if self.equal else out)))
        if self.drop > 0: out = F.dropout(out, p=self.drop, training=self.tr
        return (x if self.equal else self.shortcut(x)) + self.conv2(out)

class WideResNet(nn.Module):
    def __init__(self, depth=28, num_c=10, widen=2, drop=0.0):
        super().__init__()
        nC = [16, 16*widen, 32*widen, 64*widen]; n = (depth - 4) // 6
        self.conv1 = nn.Conv2d(3, nC[0], 3, 1, 1, bias=False)
        self.block1 = nn.Sequential(*[BasicBlock(nC[0] if i==0 else nC[1], r
        self.block2 = nn.Sequential(*[BasicBlock(nC[1] if i==0 else nC[2], r
        self.block3 = nn.Sequential(*[BasicBlock(nC[2] if i==0 else nC[3], r
        self.bn1, self.relu, self.fc = nn.BatchNorm2d(nC[3]), nn.LeakyReLU(0
    def forward(self, x):
        out = self.relu(self.bn1(self.block3(self.block2(self.block1(self.cc
        return self.fc(F.avg_pool2d(out, 8).view(x.size(0), -1))

criterion = nn.CrossEntropyLoss()

def eval_self_sup(queries):
    X_tr = torch.tensor(embeddings[queries], dtype=torch.float32).to(device)
    y_tr = torch.tensor(true_labels[queries], dtype=torch.long).to(device)
    train_dl_local = DataLoader(TensorDataset(X_tr, y_tr), batch_size=min(le
    clf = nn.Linear(embeddings.shape[1], 10).to(device)
    opt = optim.SGD(clf.parameters(), lr=2.5, momentum=0.9)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(opt, 5)
    clf.train()
    for _ in range(5):
        for inputs, targets in train_dl_local:
            opt.zero_grad(); loss = criterion(clf(inputs), targets); loss.ba
        scheduler.step()
    clf.eval(); correct = 0
    with torch.no_grad():
        for inputs, targets in test_dl:
            correct += (clf(inputs.to(device)).max(1)[1] == targets.to(devic
    return 100 * correct / len(testset)

def eval_fully_sup(queries):
    train_transform_sup = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(config['dataset']['mean'], config['dataset']['s
    ])
    sup_ds = torchvision.datasets.CIFAR10(root=config['dataset']['root'], tr
    train_dl_local = DataLoader(Subset(sup_ds, queries), batch_size=min(len(
    clf = torchvision.models.resnet18(weights=None)
    clf.conv1 = nn.Conv2d(3, 64, 3, 1, 1, bias=False)
    clf.maxpool = nn.Identity()
```

```
    clf.fc = nn.Linear(512, 10)
    clf = clf.to(device)
    opt = optim.SGD(clf.parameters(), lr=0.025, momentum=0.9, nesterov=True)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(opt, 5)
    clf.train()
    for _ in range(5):
        for inputs, targets in train_dl_local:
            opt.zero_grad(); loss = criterion(clf(inputs.to(device)), target
        scheduler.step()
    clf.eval(); correct = 0
    with torch.no_grad():
        for inputs, targets in test_loader:
            correct += (clf(inputs.to(device)).max(1)[1] == targets.to(devic
    return 100 * correct / len(testset)

def eval_semi_sup(queries):
    from torchvision.transforms import RandAugment

    weak = transforms.Compose([
        transforms.RandomCrop(32, 4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(config['dataset']['mean'], config['dataset']['s
    ])
    strong = transforms.Compose([
        transforms.RandomCrop(32, 4),
        transforms.RandomHorizontalFlip(),
        RandAugment(2, 10),
        transforms.ToTensor(),
        transforms.Normalize(config['dataset']['mean'], config['dataset']['s
    ])

    clf = WideResNet(28, 10, 2).to(device)
    opt = optim.SGD(clf.parameters(), lr=0.03, momentum=0.9, weight_decay=0.

    sup_ds = torchvision.datasets.CIFAR10(root=config['dataset']['root'], tr
    dl_l = DataLoader(Subset(sup_ds, queries), batch_size=64, shuffle=True)

    unlabeled = list(set(range(len(cifar10_full))) - set(queries))
    u_ds_weak = torchvision.datasets.CIFAR10(root=config['dataset']['root'],
    u_ds_strong = torchvision.datasets.CIFAR10(root=config['dataset']['root'
    dl_u_weak = DataLoader(Subset(u_ds_weak, unlabeled), batch_size=64, shuf
    dl_u_strong = DataLoader(Subset(u_ds_strong, unlabeled), batch_size=64,

    class_thresh = torch.ones(10).to(device) * 0.95

    NUM_ITERATIONS = 1000
    labeled_iter = iter(dl_l)
    weak_iter = iter(dl_u_weak)
    strong_iter = iter(dl_u_strong)

    for it in range(NUM_ITERATIONS):
        clf.train()
        try: x_l, y_l = next(labeled_iter)
        except StopIteration: labeled_iter = iter(dl_l); x_l, y_l = next(lab
        try: x_w, _ = next(weak_iter)
```

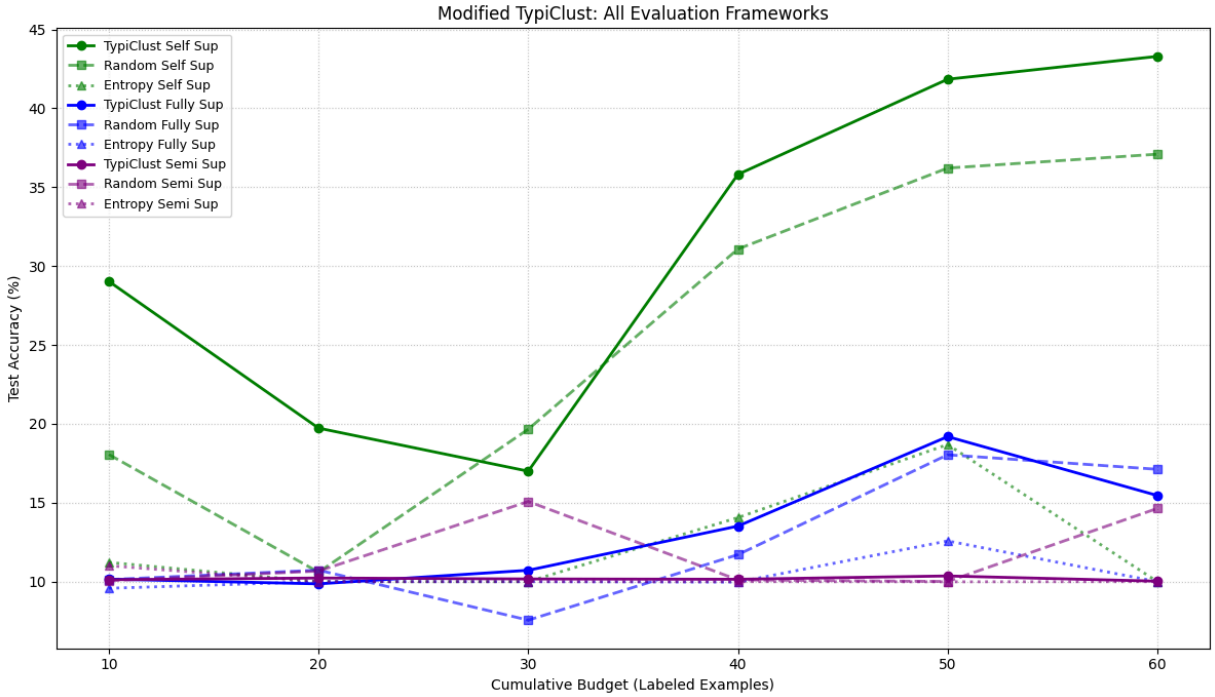
```
        except StopIteration: weak_iter = iter(dl_u_weak); x_w, _ = next(wea
        try: x_s, _ = next(strong_iter)
        except StopIteration: strong_iter = iter(dl_u_strong); x_s, _ = next

        x_l, y_l = x_l.to(device), y_l.to(device)
        x_w, x_s = x_w.to(device), x_s.to(device)

        sup_loss = nn.CrossEntropyLoss()(clf(x_l), y_l)

        with torch.no_grad():
            probs_w = F.softmax(clf(x_w), dim=1)
            max_probs, pseudo_y = probs_w.max(1)

        per_class_thresh = class_thresh[pseudo_y]
        mask = max_probs >= per_class_thresh

        for c in range(10):
            c_mask = (pseudo_y == c)
            if c_mask.sum() > 0:
                class_thresh[c] = 0.9 * class_thresh[c] + 0.1 * max_probs[c_

        if mask.sum() > 0:
            unsup_loss = nn.CrossEntropyLoss()(clf(x_s[mask]), pseudo_y[mask
        else:
            unsup_loss = torch.tensor(0.0).to(device)

        loss = sup_loss + unsup_loss
        opt.zero_grad(); loss.backward(); opt.step()

    clf.eval(); correct = 0
    with torch.no_grad():
        for x, y in test_loader:
            correct += (clf(x.to(device)).max(1)[1] == y.to(device)).sum().i
    return 100 * correct / len(testset)
```

In [6]:

```
budgets = [10, 20, 30, 40, 50, 60]
results = {'self_sup': {'tc': [], 'rand': [], 'ent': []}, 'fully_sup': {'tc'

for b in budgets:
    print(f"\nEvaluating Budget {b}")
    tc_q = get_typiclust_queries(embeddings, B=b, max_clusters=config['typic
    rand_q = random.sample(range(len(embeddings)), b)
    ent_q = get_entropy_queries(embeddings, true_labels, B=b)

    results['self_sup']['tc'].append(eval_self_sup(tc_q))
    results['self_sup']['rand'].append(eval_self_sup(rand_q))
    results['self_sup']['ent'].append(eval_self_sup(ent_q))

    results['fully_sup']['tc'].append(eval_fully_sup(tc_q))
    results['fully_sup']['rand'].append(eval_fully_sup(rand_q))
    results['fully_sup']['ent'].append(eval_fully_sup(ent_q))

    results['semi_sup']['tc'].append(eval_semi_sup(tc_q))
    results['semi_sup']['rand'].append(eval_semi_sup(rand_q))
    results['semi_sup']['ent'].append(eval_semi_sup(ent_q))
```

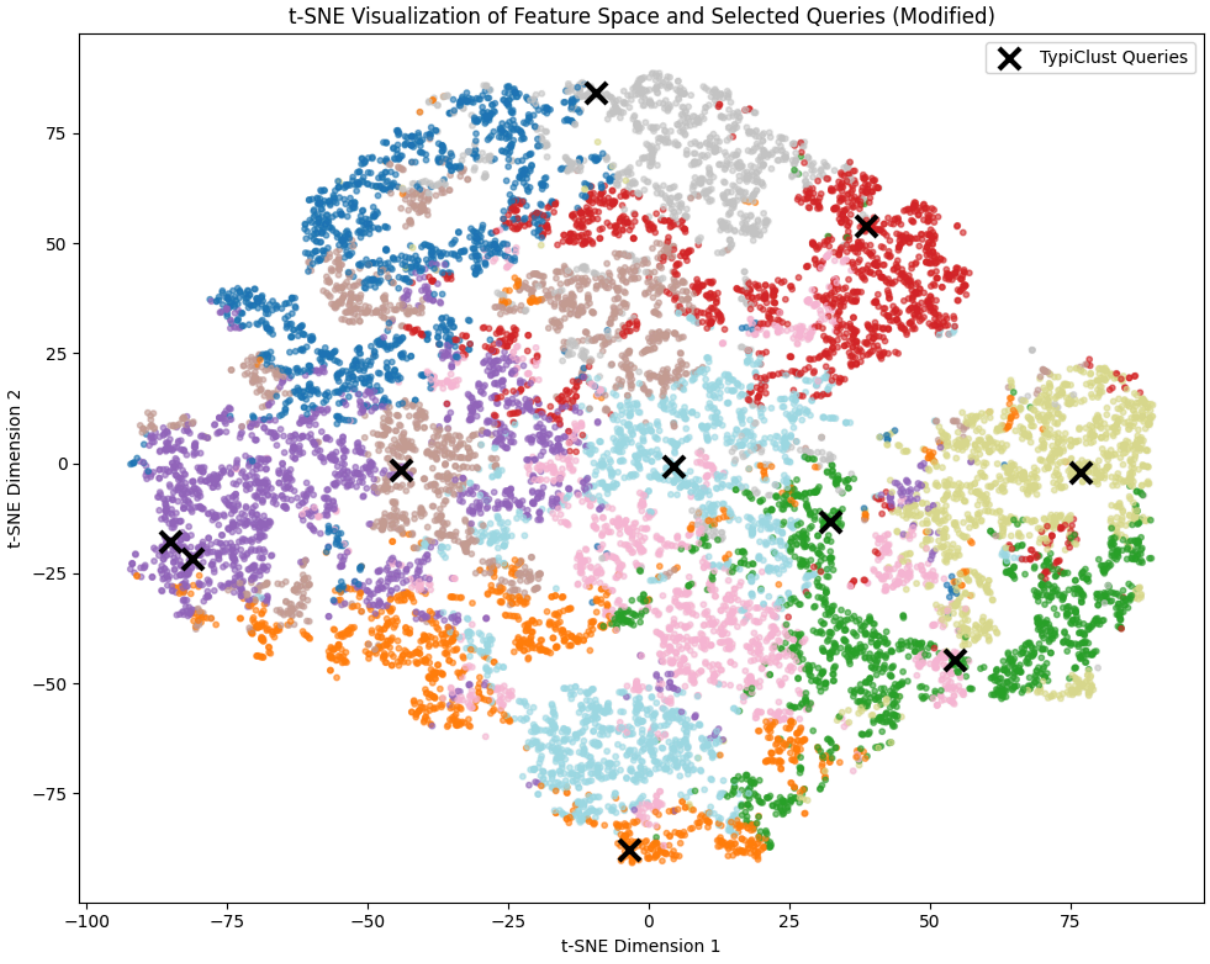
```
plt.figure(figsize=(12, 7))
colors = {'fully_sup': 'blue', 'self_sup': 'green', 'semi_sup': 'purple'}
labels = {'fully_sup': 'Fully Sup', 'self_sup': 'Self Sup', 'semi_sup': 'Sem

for k in results:
    plt.plot(budgets, results[k]['tc'], marker='o', label=f'TypiClust {label
    plt.plot(budgets, results[k]['rand'], marker='s', label=f'Random {labels
    plt.plot(budgets, results[k]['ent'], marker='^', label=f'Entropy {labels

plt.title('Modified TypiClust: All Evaluation Frameworks')
plt.xlabel('Cumulative Budget (Labeled Examples)')
plt.ylabel('Test Accuracy (%)')
plt.legend(fontsize=9)
plt.grid(True, linestyle=':', alpha=0.7)
plt.tight_layout()
plt.savefig(os.path.join(results_dir, 'learning_curve_modified_combined.pdf'
plt.show()
```

```
Evaluating Budget 10

Evaluating Budget 20

Evaluating Budget 30

Evaluating Budget 40

Evaluating Budget 50

Evaluating Budget 60
```




In [7]:
```
subset_idx = np.arange(15000)
emb_subset = embeddings[subset_idx]

tsne = TSNE(n_components=2, random_state=42)
emb_2d = tsne.fit_transform(emb_subset)
```

```
kmeans_viz = KMeans(n_clusters=10, random_state=42, n_init=10)
cluster_labels_viz = kmeans_viz.fit_predict(emb_subset)

tc_queries_viz = get_typiclust_queries(emb_subset, B=10, max_clusters=10, nc

plt.figure(figsize=(10, 8))
plt.scatter(emb_2d[:, 0], emb_2d[:, 1], c=cluster_labels_viz, cmap='tab20',

query_2d = emb_2d[tc_queries_viz]
plt.scatter(query_2d[:, 0], query_2d[:, 1], c='black', marker='x', s=150, li

plt.title('t-SNE Visualization of Feature Space and Selected Queries (Modifi
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(results_dir, 'tsne_modified_final.pdf'))
plt.show()
```



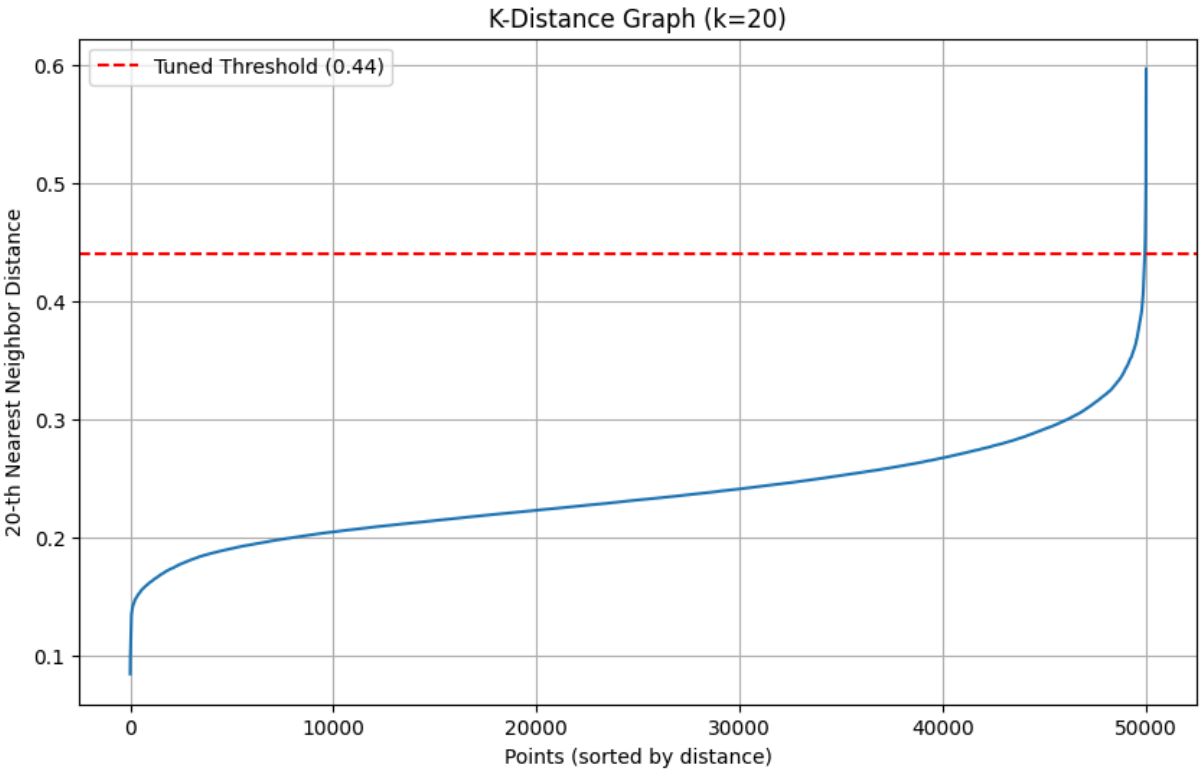

In [8]:

```
k_values = [5, 10, 20]

plt.figure(figsize=(10, 6))

for k in k_values:
    nn = NearestNeighbors(n_neighbors=k)
    nn.fit(embeddings)
```

```
    distances, _ = nn.kneighbors(embeddings)
    k_distances = np.sort(distances[:, -1])

    plt.plot(k_distances, label=f'k={k}')

plt.title('K-Distance Graph for Outlier Detection')
plt.xlabel('Data Points (sorted by distance to k-th neighbor)')
plt.ylabel('k-th Nearest Neighbor Distance')
plt.legend()
plt.grid(True)

plt.show()
```


In [9]:
```
k = 20

nn = NearestNeighbors(n_neighbors=k)
nn.fit(embeddings)

distances, indices = nn.kneighbors(embeddings)
k_distances = np.sort(distances[:, k-1])

plt.figure(figsize=(10, 6))
plt.plot(k_distances)

plt.axhline(y=0.31, color='r', linestyle='--', label='Tuned Threshold (0.31)')

plt.title(f'K-Distance Graph (k={k})')
plt.xlabel('Points (sorted by distance)')
plt.ylabel(f'{k}-th Nearest Neighbor Distance')

plt.legend()
```

```
plt.grid(True)
plt.show()

print(f"Median k-distance: {np.median(k_distances):.4f}")
```




```
Median k-distance: 0.2317
```